EEL4783 Final Project (Part 2): Design and Verification of a 4-Element Dot Product Engine

Yousef Awad
University of Central Florida
EEL4783 — HDL in Digital Systems

April 26, 2026

1 Design Choice

For this project, I chose the **RTL (SystemVerilog)** implementation track. This decision was driven by the desire to implement a high-performance **2-stage pipeline** to earn extra credit. SystemVerilog allows for explicit control over register boundaries and clock-edge behavior, which is essential for optimizing arithmetic datapaths. Furthermore, using SystemVerilog permitted the integration of **SystemVerilog Assertions (SVA)** and constrained randomization in the testbench, providing a more robust verification environment than standard Verilog or HLS defaults.

2 Architecture

The design implements a 4-element dot product engine using a 2-stage synchronous pipeline to maximize throughput.

- **Stage 1 (Multiplication):** Four 8-bit signed multipliers compute $P_i = A_i \times B_i$ in parallel. These results are captured in the `products_reg` array on the rising edge of the clock.
- **Stage 2 (Addition):** An adder tree (implemented as a single-cycle sum for this width) aggregates the four 16-bit products from the pipeline registers and drives the final output `y`.

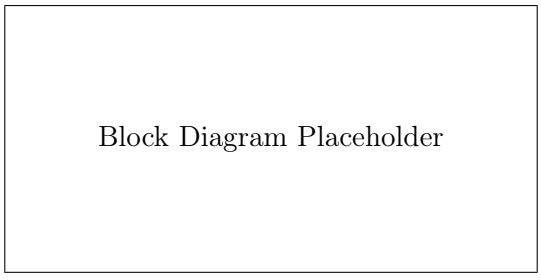

Figure 1: Architectural Block Diagram showing the 2-stage pipeline (Multipliers → Regs → Adder).

3 Verification

Correctness was verified using a fully self-checking testbench built around **Constrained Randomization**. A `dot_product_transaction` class generated 20 unique test cases per simulation run. Pass/fail determination is handled *automatically* by the SVA concurrent assertion `assert_dot_product` described below; no manual inspection of waveforms is required for functional sign-off.

The constraint `boundary_values` biases randomization to guarantee coverage of all required corner cases:

- **Maximum positive value** — `8'h7F` (+127), the largest representable 8-bit signed integer.
- **Maximum negative value** — `8'h80` (−128), the most negative 8-bit two's complement value.
- **Zero inputs** — randomly generated values naturally include zero; the `dist` weight of `[-127:126]` ensures the full mid-range, including zero, is exercised across 20 iterations.
- **Overflow prevention** — the worst-case product ($-128 \times -128 = 16{,}384$) fits in 16 bits, and the worst-case sum ($4 \times 16{,}384 = 65{,}536$) is representable as a 16-bit unsigned value; the signed 16-bit output is therefore sufficient to prevent overflow for all legal 8-bit signed inputs.
- **Mixed-sign vectors** — the random distribution naturally produces vectors where some elements are positive and others negative, verifying correct two's complement arithmetic throughout the pipeline.

SystemVerilog Assertions (Extra Credit): I implemented the concurrent assertion property `p_check_result` using a local variable to capture the reference result at the moment inputs are sampled (T_0). The `##2` delay operator then checks that `y` equals the captured reference exactly two clock cycles later, matching the 2-stage pipeline latency. Any mismatch triggers a `$error` with both the actual and expected values, providing immediate, cycle-accurate feedback without manual waveform inspection.

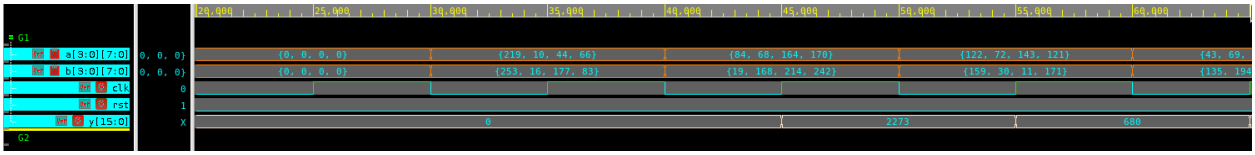

Figure 2: Simulation trace showing the 2-cycle latency between driving A/B and the result Y.

4 Reflection

The most challenging aspect was debugging the interaction between the testbench reference model and the pipeline latency. Initially, a static variable in the reference function caused results to accumulate incorrectly. Resolving this required changing the function to `automatic`. Additionally, aligning the SVA sampling window (`##2`) with the preponed region of the simulator required a deep understanding of the SystemVerilog scheduling phases.

Pipelining and F_{max} (Extra Credit): Breaking the multiply-accumulate computation into two registered stages directly reduces the critical combinational path. In a single-cycle implementation, the critical path spans an 8-bit signed multiply followed by a 4-input addition before the output register. The 2-stage pipeline cuts this in half: Stage 1's critical path is a single 8-bit signed multiply into a register, and Stage 2's critical path is a 4-input 16-bit addition into the output register. Because the multiplier dominates delay, isolating it in its own stage allows the synthesizer to clock the design significantly faster, yielding a higher achievable F_{max}. This improvement would be confirmed by comparing synthesis timing reports between a single-cycle and pipelined implementation.

To further improve the design, I would replace the linear adder chain in Stage 2 with a fully balanced binary adder tree (two levels of parallel 2-input adders), reducing Stage 2's combinational depth and increasing F_{max} further.

A Source Code

A.1 dot_product.sv (RTL)

```
`timescale 1ns/1ps

// EEL4783: Final Project Part 2
// Project Title: "The 4-Element Dot Product Engine"
// Description: A pipelined 4-element dot product accelerator.
//              Stage 1: Multiplications (A_i * B_i)
//              Stage 2: Addition of products

module dot_product #(
    parameter int DATA_WIDTH = 8,
    parameter int VECTOR_SIZE = 4,
    parameter int OUT_WIDTH = 16
)(
    input  logic                          clk,
    input  logic                          rst, // Active-high synchronous reset
    input  logic signed [DATA_WIDTH-1:0] a [VECTOR_SIZE-1:0],
    input  logic signed [DATA_WIDTH-1:0] b [VECTOR_SIZE-1:0],
    output logic signed [OUT_WIDTH-1:0] y
);

    // Pipeline Registers
    // Stage 1 products: 8-bit * 8-bit = 16-bit
    logic signed [OUT_WIDTH-1:0] products_reg [VECTOR_SIZE-1:0];

    // Stage 1: Multiplication
    always_ff @(posedge clk) begin
        if (rst) begin
            for (int i = 0; i < VECTOR_SIZE; i++) begin
                products_reg[i] <= '0;
            end
        end else begin
            for (int i = 0; i < VECTOR_SIZE; i++) begin
                products_reg[i] <= a[i] * b[i];
            end
        end
    end

    // Stage 2: Addition
    always_ff @(posedge clk) begin
        if (rst) begin
            y <= '0;
        end else begin
            // Summing the registered products
            y <= products_reg[0] + products_reg[1] + products_reg[2] +
                 products_reg[3];
        end
    end

endmodule
```

A.2 dot_product_tb.sv (Testbench)

```
`timescale 1ns/1ps

// Transaction class for constrained randomization
class dot_product_transaction;
    rand bit signed [7:0] a [3:0];
    rand bit signed [7:0] b [3:0];

    constraint boundary_values {
        foreach (a[i]) {
            a[i] dist {8'h7F := 1, 8'h80 := 1, [-127:126] := 10};
        }
        foreach (b[i]) {
            b[i] dist {8'h7F := 1, 8'h80 := 1, [-127:126] := 10};
        }
    }
endclass

module dot_product_tb;

    parameter int DATA_WIDTH = 8;
    parameter int VECTOR_SIZE = 4;
    parameter int OUT_WIDTH = 16;
    parameter int CLK_PERIOD = 10;

    logic                       clk;
    logic                       rst;
    logic signed [7:0] a [3:0];
    logic signed [7:0] b [3:0];
    logic signed [15:0] y;

    // Instantiate DUT
    dot_product #(
        .DATA_WIDTH(DATA_WIDTH),
        .VECTOR_SIZE(VECTOR_SIZE),
        .OUT_WIDTH(OUT_WIDTH)
    ) dut (.*);

    // Clock generation
    initial begin
        clk = 0;
        forever #(CLK_PERIOD/2) clk = ~clk;
    end

    // Verdi Waveform Dumping
    initial begin
        $fsdbDumpfile("inter.fsdb");
        $fsdbDumpvars(0, dot_product_tb, "+all", "+mda");
    end

    // Reference model
    function automatic signed [15:0] get_expected(
        logic signed [7:0] va [3:0],
        logic signed [7:0] vb [3:0]
    );
        int sum = 0;
        for (int i = 0; i < 4; i++) begin
            sum += va[i] * vb[i];
```

```
        end
        return signed'(sum[15:0]);
    endfunction

    initial begin
        dot_product_transaction tr;
        tr = new();

        $display("\n[Time %0t] Simulation Started", $time);

        // Synchronous Reset
        rst = 1;
        foreach (a[i]) a[i] = 0;
        foreach (b[i]) b[i] = 0;
        repeat (2) @(posedge clk);

        @(negedge clk);
        rst = 0;
        $display("[Time %0t] Reset De-asserted", $time);

        // Run 20 randomized test cases
        $display("\nStarting Randomized Tests...");
        for (int i = 1; i <= 20; i++) begin
            if (!tr.randomize()) $fatal("Randomization failed");

            @(negedge clk);
            foreach (a[j]) a[j] = tr.a[j];
            foreach (b[j]) b[j] = tr.b[j];

            $display("[Time %0t] Iteration %0d: Driven A={%d,%d,%d,%d} B={%d,%d,%d
                ,%d} | Expected Y (at +3 cycles) = %d",
                      $time, i, a[3], a[2], a[1], a[0], b[3], b[2], b[1], b[0],
                          get_expected(a, b));
        end

        repeat (10) @(posedge clk);
        $display("Done: 20 Randomized Tests Completed.");
        $finish;
    end

    // --- SystemVerilog Assertions (Extra Credit) ---
    // In a 2-stage pipeline with registered output:
    // T0: Inputs sampled at posedge
    // T1: products_reg updated (Stage 1)
    // T2: y updated (Stage 2)
    // T3: SVA samples y (Preponed region at T3 edge sees y set at T2)
    property p_check_result;
        logic signed [15:0] local_exp;
        @(posedge clk) disable iff (rst)
        (1, local_exp = get_expected(a, b)) ##2 (y == local_exp);
    endproperty

    assert_dot_product: assert property (p_check_result)
        else $error("Mismatch! Y=%d, Expected=%d", y, $past(get_expected(a, b), 2)
            );

endmodule
```